Assignment -12.3

Name: Akshaya Nemalipuri

Batch: 37

Roll No: 2303A2441

Lab: 12.3

Task 1: Sorting Student Records for Placement Drive
Scenario
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
Tasks
1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.
Expected Outcome
• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

Prompt:-

write the python code on SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by

CGPA in descending order with example usage .

#Write a Python program to sort the student records based on CGPA in descending order.# Define a list of student records, where each record is a dictionary

Code:-

```python
lab_12.3.1.py > ...
# write the python code on SR University's Training and Placement Cell needs to shortlist candidates efficiently
# CGPA in descending order with example usage .
#Write a Python program to sort the student records based on CGPA in descending order.# Define a list of student
students = [
    {"name": "Alice", "cgpa": 3.8},
    {"name": "Bob", "cgpa": 3.5},
    {"name": "Charlie", "cgpa": 3.9},
    {"name": "David", "cgpa": 3.6}
]
# Sort the student records by CGPA in descending order
sorted_students = sorted(students, key=lambda x: x["cgpa"], reverse=True)
# Print the sorted student records
print("Sorted student records by CGPA (descending order):")
for student in sorted_students:
    print(f"Name: {student['name']}, CGPA: {student['cgpa']}")
# Example usage
if __name__ == "__main__":
    # The sorted student records will be printed when the program is run
    pass
expected_output = """Sorted student records by CGPA (descending order):
Name: Charlie, CGPA: 3.9
Name: Alice, CGPA: 3.8
Name: David, CGPA: 3.6
```

```
PS C:\Users\sc\Downloads\AI_Assiant_code-lab> python -u "c:\Users\sc\Down
Sorted student records by CGPA (descending order):
Name: Charlie, CGPA: 3.9
Name: Alice, CGPA: 3.8
Name: Bob, CGPA: 3.5
```

Explain code:-

At SR University, the program stores student details in a list of dictionaries and uses Python's built-in `sorted()` function with `key=lambda x: x["cgpa"]` and `reverse=True` to arrange the records in descending order based on CGPA.

The sorted list is then printed using a `for` loop, displaying students from highest to lowest CGPA for efficient shortlisting during placements.

Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.
• Instructions:
• Students implement Bubble Sort normally.
• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
• Request AI to provide time complexity analysis.
• Expected Output:
• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt:-

Write a Python implementation of Bubble Sort Students implement Bubble Sort normally. generate inline comments explaining key logic (like swapping, passes, and termination time complexity analysis

Code:-

lab_12.3.2.py > ...

```python
# Write a Python implementation of Bubble Sort Students implement Bubble Sort normally. generate inline comments
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  # Swap
                swapped = True  # Set the flag to True if a swap occurred
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break  # Terminate early if the array is already sorted
# Example usage
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    bubble_sort(arr)
    print("Sorted array:", arr)
# Time complexity analysis:
```

```
lab_12.3.2.py > ...
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    bubble_sort(arr)
    print("Sorted array:", arr)
# Time complexity analysis:
# The worst-case and average-case time complexity of Bubble Sort is O(n^2) because
# it requires two nested loops to compare each element with every other element.
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\sc\Downloads\AI_Assiant_code-lab> python -u "c:\Users\sc\Downloads\AI_Assiant_code-lab\lab_12.3.2.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\sc\Downloads\AI_Assiant_code-lab>
```

Explain code:-

Bubble Sort works by repeatedly comparing two neighboring elements and swapping them if they are in the wrong order, so the largest values slowly “bubble up” to the end after each pass. If during a full pass no swaps happen, it means the array is already sorted, so the algorithm stops early to save time.

Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted

lists.

• Expected Output:

• Working Quick Sort and Merge Sort implementations.

• AI-generated explanation of average, best, and worst-case

complexities.

Prompt:-

#Implement Quick Sort and Merge Sort using recursion explanation of average, best, and worst-case complexities.

Quick Sort Implementation

code:-

lab_12.3.3.py > ...

```python
#Implement Quick Sort and Merge Sort using recursion explanation of average, best, and worst-case complexities.
# Quick Sort Implementation
def quick_sort(arr, low, high):
    if low < high:
        # Partition the array and get pivot index
        pi = partition(arr, low, high)
        # Recursively sort elements before and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):
    # Choose the rightmost element as pivot
    pivot = arr[high]
    # Index of smaller element (indicates right position of pivot)
    i = low - 1
    for j in range(low, high):
        # If current element is smaller than or equal to pivot
        if arr[j] <= pivot:
            i += 1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\sc\Downloads\AI_Assiant_code-lab\lab_11.2.4.py

```python
def partition(arr, low, high):
            arr[i], arr[j] = arr[j], arr[i]  # Swap elements
    # Place pivot in its correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Merge Sort Implementation
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2  # Find the middle point
        left_half = arr[:mid]   # Divide the array into two halves
        right_half = arr[mid:]
        merge_sort(left_half)   # Recursively sort first half
        merge_sort(right_half)  # Recursively sort second half
        merge(arr, left_half, right_half)

def merge(arr, left_half, right_half):
    i, j, k = 0, 0, 0
    while i < len(left_half) and j < len(right_half):
```

lab_12.3.3.py > partition

```python
def merge(arr, left_half, right_half):
        if left_half[i] < right_half[j]:
            arr[k] = left_half[i]
            i += 1
        else:
            arr[k] = right_half[j]
            j += 1
        k += 1

    while i < len(left_half):
        arr[k] = left_half[i]
        i += 1
        k += 1

    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1
```

lab_12.3.3.py > partition

```python
# Time Complexity Analysis:
# Quick Sort:
# Best Case: O(n log n) - when the pivot divides the array into two nearly equal halves.
# Average Case: O(n log n) - on average over all possible inputs.
# Worst Case: O(n^2) - when the pivot is always the smallest or largest element (e.g., already sorted array).
#
# Merge Sort:
# Best Case: O(n log n) - always maintains O(n log n) time complexity regardless of input.
# Average Case: O(n log n)
# Worst Case: O(n log n)
# Example usage
if __name__ == "__main__":
    arr_quick = [10, 7, 8, 9, 1, 5]
    print("Original array for Quick Sort:", arr_quick)
    quick_sort(arr_quick, 0, len(arr_quick) - 1)
    print("Sorted array using Quick Sort:", arr_quick)

    arr_merge = [38, 27, 43, 3, 9, 82, 10]
```

C:\Users\sc\Downloads\AI_Assiant_code-lab\lab_11.2.4.py

```python
    print("Original array for Quick Sort:", arr_quick)
    quick_sort(arr_quick, 0, len(arr_quick) - 1)
    print("Sorted array using Quick Sort:", arr_quick)

    arr_merge = [38, 27, 43, 3, 9, 82, 10]
    print("Original array for Merge Sort:", arr_merge)
    merge_sort(arr_merge)
    print("Sorted array using Merge Sort:", arr_merge)
```

```
Sorted array using Quick Sort: [1, 5, 7, 8, 9, 10]
t_code-lab\lab_12.3.3.py"
Original array for Quick Sort: [10, 7, 8, 9, 1, 5]
Original array for Quick Sort: [10, 7, 8, 9, 1, 5]
Sorted array using Quick Sort: [1, 5, 7, 8, 9, 10]
Original array for Merge Sort: [38, 27, 43, 3, 9, 82, 10]
Original array for Merge Sort: [38, 27, 43, 3, 9, 82, 10]
Sorted array using Merge Sort: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\sc\Downloads\AI_Assiant_code-lab>
```

Explain code:-

Quick Sort works by choosing a pivot element, placing it in its correct position, and recursively sorting the smaller elements on the left and larger elements on the right, making it very fast on average but slow ($O(n^2)$) when partitions are

highly unbalanced. Merge Sort divides the array into smaller halves, recursively sorts them, and then merges them back together in order, consistently giving O(n log n) time in best, average, and worst cases.

Task 4 (Real-Time Application – Inventory Management System)
Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.
Task:
• Use AI to suggest the most efficient search and sort algorithms for this use case.
• Implement the recommended algorithms in Python.
• Justify the choice based on dataset size, update frequency, and performance requirements.
Expected Output:
• A table mapping operation → recommended algorithm → justification.
• Working Python functions for searching and sorting the inventory.

Prompt:-

A retail store's inventory system contains thousands of products,each with attributes like product ID, name, price, and stock quantity. Store staff need to: Quickly search for a product by ID or name. Sort products by price or quantity for stock analysis

#Write a Python program to implement a product inventory system that allows searching and sorting of products based on the given attributes. Include example usage demonstrating the search and sort functionalities.

Code:-

lab_12.3.4.py > ...

```python
#A retail store's inventory system contains thousands of products,each with attributes like product ID, name, pri
#Write a Python program to implement a product inventory system that allows searching and sorting of products bas
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity
class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        """Add a product to the inventory"""
        self.products.append(product)

    def search_by_id(self, product_id):
        """Search for a product by ID"""
        for product in self.products:
            if product.product_id == product_id:
                return product
        return None

```

lab_12.3.4.py > ...

```python
class Inventory:
    def search_by_name(self, name):
        """Search for products by name"""
        return [product for product in self.products if product.name.lower() == name.lower()]

    def sort_by_price(self):
        """Sort products by price"""
        return sorted(self.products, key=lambda x: x.price)

    def sort_by_stock_quantity(self):
        """Sort products by stock quantity"""
        return sorted(self.products, key=lambda x: x.stock_quantity)
# Example usage
if __name__ == "__main__":
    inventory = Inventory()
    inventory.add_product(Product(1, "Laptop", 999.99, 10))
    inventory.add_product(Product(2, "Smartphone", 499.99, 20))
    inventory.add_product(Product(3, "Headphones", 199.99, 15))

    # Search for a product by ID
    product = inventory.search_by_id(2)
    if product:
        print(f"Product found: {product.name}, Price: {product.price}, Stock: {product.stock_quantity}")
```

```python
lab_12.3.4.py > ...
    else:
        print("Product not found")

    # Search for products by name
    products = inventory.search_by_name("Laptop")
    for product in products:
        print(f"Product found: {product.name}, Price: {product.price}, Stock: {product.stock_quantity}")

    # Sort products by price
    sorted_by_price = inventory.sort_by_price()
    print("Products sorted by price:")
    for product in sorted_by_price:
        print(f"{product.name}: ${product.price}")

    # Sort products by stock quantity
    sorted_by_stock = inventory.sort_by_stock_quantity()
    print("Products sorted by stock quantity:")
    for product in sorted_by_stock:
        print(f"{product.name}: Stock {product.stock_quantity}")
```

```
PS C:\Users\sc\Downloads\AI_Assiant_code-lab> python -u "c:\Users\sc\Downloads\AI_Assiant_code-lab\lab_12.3.4.py"
Product found: Smartphone, Price: 499.99, Stock: 20
Product found: Laptop, Price: 999.99, Stock: 10
Products sorted by price:
Headphones: $199.99
Smartphone: $499.99
Laptop: $999.99
Products sorted by stock quantity:
Laptop: Stock 10
Headphones: Stock 15
Smartphone: Stock 20
PS C:\Users\sc\Downloads\AI_Assiant_code-lab>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Prompt:-

An AI-powered FinTech Lab at SR University is building a tool for analyzing tock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently. Optimize sorting with Heap Sort and searching with Hash Maps. Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Code:-

```
#An AI-powered FinTech Lab at SR University is building a tool for
analyzing tock price movements. The requirement is to quickly sort stocks
by daily gain/loss and search for specific stock symbols efficiently.
Optimize sorting with Heap Sort and searching with Hash Maps. Compare
performance with standard library functions (sorted(), dict lookups) and
analyze trade-offs.
import heapq
class Stock:
    def __init__(self, symbol, daily_gain_loss):
        self.symbol = symbol
        self.daily_gain_loss = daily_gain_loss

    def __lt__(self, other):
        return self.daily_gain_loss < other.daily_gain_loss
class StockAnalyzer:
    def __init__(self):
        self.stocks = []
        self.stock_map = {}

    def add_stock(self, symbol, daily_gain_loss):
        stock = Stock(symbol, daily_gain_loss)
        heapq.heappush(self.stocks, stock)  # Add stock to the heap
        self.stock_map[symbol] = stock  # Add stock to the hash map

    def get_top_gainers_losers(self, n=5):
        """Get top n gainers and losers"""
        top_gainers = heapq.nlargest(n, self.stocks)
        top_losers = heapq.nsmallest(n, self.stocks)
        return top_gainers, top_losers

    def search_stock(self, symbol):
        """Search for a stock by symbol"""
        return self.stock_map.get(symbol, None)  # O(1) lookup in hash map
# Example usage
```

```python
if __name__ == "__main__":
    analyzer = StockAnalyzer()
    analyzer.add_stock("AAPL", 2.5)
    analyzer.add_stock("GOOGL", -1.2)
    analyzer.add_stock("AMZN", 3.0)
    analyzer.add_stock("MSFT", -0.5)
    analyzer.add_stock("TSLA", 4.0)

    top_gainers, top_losers = analyzer.get_top_gainers_losers()
    print("Top Gainers:")
    for stock in top_gainers:
        print(f"{stock.symbol}: {stock.daily_gain_loss}%")

    print("\nTop Losers:")
    for stock in top_losers:
        print(f"{stock.symbol}: {stock.daily_gain_loss}%")

    # Search for a specific stock
    stock = analyzer.search_stock("GOOGL")
    if stock:
        print(f"\nStock found: {stock.symbol}, Daily Gain/Loss:
{stock.daily_gain_loss}%")
    else:
        print("\nStock not found")
```

Explain code:-

At SR University's FinTech Lab, this program uses a heap (Heap Sort concept) to efficiently track top gainers and losers while storing stocks in a hash map (dictionary) for instant O(1) symbol lookup. Compared to `sorted()` which takes O(n log n) each time and normal list searching which takes O(n), the heap and dictionary combination is faster for frequent updates and searches, though it uses slightly more memory to maintain both data structures.